# EX15b - NumPy Practice - Shape, Slicing and Broadcasting

Beginner Step-by-Step Guide

This guide follows the real file `ex15b_Numpy_shapes_and_operations_practice.py`. It explains the problem first and then the code from the perspective of someone starting Python.

## 1. What we want to solve

Practice array geometry and basic NumPy operations before applying them to larger datasets.

### Problem variables / elements

| Name | Meaning |
|---|---|
| vector_a | 1D array with 3 elements |
| matrix_a | 2D array with shape (2,3) |

### Equation or logic on paper

```
shape / ndim / size / dtype
broadcasting
reshape / transpose
vstack / hstack
* vs @
```

### Manual check

matrix_a has shape (2,3), ndim=2 and size=6. vector_a has shape (3,), ndim=1 and size=3.

## 2. What we already know and what is new

**Already seen:**

- np.array
- shape
- basic slicing

**New in this exercise:**

- ndim
- size
- dtype
- 2D slicing
- element-wise operations
- broadcasting
- reshape
- transpose
- vstack/hstack
- axis
- argmax
- dot/cross
- @

# 3. New Python concepts

## 1. ndim

**Syntax:** `A.ndim`

Counts structural axes. vector_a[i] uses one index; matrix_a[i,j] uses two.

## 2. Broadcasting

**Syntax:** `matrix_a + offset`

NumPy compares shapes from the right. (2,3) and (3,) are compatible because the last 3 matches.

## 3. reshape

**Syntax:** `vector_a.reshape(1,3)`

Changes structure without changing the three data values.

## 4. axis

**Syntax:** `np.max(matrix_b, axis=0)`

axis=0 reduces rows leaving one result per column; axis=1 reduces columns leaving one per row.

## 5. Matrix product

**Syntax:** `A @ v`

@ performs matrix algebra; * multiplies element by element.

# 4. Code step by step

## Step 1. Inspect geometry

```
vector_a.shape
vector_a.ndim
vector_a.size
vector_a.dtype
```

Before operating it helps to know shape, axes, number of elements and dtype.

## Step 2. Index

```
matrix_a[0, 0]
matrix_a[0, :]
matrix_a[:, 1]
matrix_a[:, 1:]
```

Two indices mean row and column. : represents all values or a range.

## Step 3. Element-wise operations

```
vector_a + vector_b
vector_a * vector_b
vector_a**2
```

NumPy applies each operator to corresponding positions.

## Step 4. Broadcasting

```
offset = np.array([100.0, 200.0, 300.0])
result = matrix_a + offset
```

The shape-(3,) vector is added to each row of the (2,3) matrix.

## Step 5. reshape and T

```
row = vector_a.reshape(1,3)
column = vector_a.reshape(3,1)
matrix_a_T = matrix_a.T
```

Same data, different structures. T swaps rows and columns.

## Step 6. Stack

```
np.vstack((matrix_a, new_row))
```

```
np.hstack((matrix_a, new_column))
```

vstack needs compatible columns; hstack needs compatible rows.

## Step 7. Reduce by axes

```
np.max(matrix_b, axis=0)
np.max(matrix_b, axis=1)
```

The specified axis disappears from the reduction result.

## Step 8. Vector algebra

```
vector_a @ vector_b
np.cross(vector_a, vector_b)
matrix_a @ vector_b
```

The exercise compares dot, cross and matrix-vector products.

## 5. Execution flow

```
create arrays
 -> inspect geometry
 -> index/slice
 -> element-wise ops
 -> broadcasting
 -> reshape/stack
 -> axis reductions
 -> algebra
```

## 6. Expected result / check

Among other results: vector_a@vector_b=32, cross=[-3,6,-3], matrix_a@vector_b=[32,77].

## 7. Common beginner mistakes

- Confusing ndim with physical dimension.
- Thinking (3,) and (1,3) are the same.
- Thinking * is matrix multiplication.
- Not checking shapes before broadcasting.
- Treating axis as physical x/y/z.

## 8. If you remember only five things

- shape gives axis lengths.
- ndim counts structural axes.
- Broadcasting compares shapes from the right.
- reshape changes shape, not data.
- * is element-wise and @ is matrix algebra.

## 9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

## Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.